StockSync

Product Batch Traceability and Recall Management System

DBMS Level 3 Project Report

PES University

Department of Computer Science and Engineering

Team Member	SRN
Rekha Dhorigol	PES1UG24CS370
Reet Porwal	PES1UG24CS369

Section: G

Repository: https://github.com/reetporwal66/inventory-traceability

1. Abstract

StockSync is a web-based product batch traceability and recall management system. It connects inventory operations with quality and recall workflows, including products, supplier batches, warehouse stock, reservations, transfers, shipments, defect reports, recall cases and recovery actions. MySQL stores structured operational records and enforces relational integrity. MongoDB stores event documents synchronized from a MySQL event outbox. The Next.js application provides role-aware access for administrators, inventory staff and quality staff. The project's aim is to support the traceable movement of stock and help identify affected batches and downstream recipients when a quality issue requires a recall.

2. Problem Statement and Objectives

Inventory traceability becomes difficult when stock is distributed across warehouses, reserved for orders, transferred, shipped, or quarantined after a quality issue. A recall process must identify affected batches, their current stock locations and relevant downstream recipients while preventing unsafe stock from being treated as available.

- Track products and supplier batches through warehouses and shipments.
- Track on-hand, reserved, quarantined and derived available quantities.
- Manage reservations, transfers, shipment progress, discrepancies, defects and recalls.
- Enforce data integrity using primary and foreign keys, constraints, indexes and transactional database operations.
- Separate permissions by user role.
- Synchronize operational inventory events from MySQL to MongoDB through an outbox workflow.

3. Functional Requirements

Requirement	Description
Authentication and authorization	Role-aware sign-in and permissions for ADMIN, INVENTORY_STAFF and QUALITY_STAFF.
Inventory and batch traceability	View stock by product, batch and warehouse; distinguish on-hand, reserved, quarantined and available quantities.
Reservations and transfers	Reserve/cancel stock and record transfers between warehouses.
Shipment management	Track shipment status, dispatched and received quantities, and discrepancies.
Quality and recalls	Record defects, initiate recall cases, link affected batches, quarantine stock and record recovery actions.
Recipient traceability	Connect shipment items and batches to recipients for downstream traceability.
Event synchronization	Write selected inventory events to a MySQL outbox and synchronize event documents to MongoDB.

4. Technology Stack and Database Justification

Component	Technology
Frontend/application	Next.js, React and TypeScript
Relational database	MySQL/InnoDB
Document database	MongoDB
Authentication/session	bcryptjs and jose (as documented in the repository)
Input validation	Zod
Database connectivity	mysql2 and mongodb
Version control	Git and public GitHub repository

MySQL is used for structured entities and relationships that benefit from referential integrity, constraints, joins and transactions. MongoDB is used for event-oriented documents whose payload can vary by event type. The project uses a document database and does not claim to implement vector embeddings or vector similarity search.

5. Database Design: Entities and Relationships

The repository SQL setup script defines the following 16 relational tables: app_users, products, suppliers, batches, warehouses, inventory, stock_reservations, stock_transfers, recipients, shipments, shipment_items, defect_reports, recalls, recall_batches, recall_recoveries and event_outbox. The ER diagram represents the entities used in the StockSync database and their relationships. The relational schema provides the corresponding table structure. The SQL setup script defines the table columns, keys, constraints and relationships implemented in the database.

- products 1—many batches; suppliers 1—many batches.
- batches and warehouses connect to inventory; a unique (batch_id, warehouse_id) key prevents duplicate stock rows for the same batch and warehouse.
- stock_reservations reference a batch, warehouse and optional user; stock_transfers reference a batch, source warehouse, destination warehouse and optional user.
- recipients 1—many shipments; shipments 1—many shipment_items; each shipment item references a batch.
- defect_reports reference batches and may reference reporting/reviewing users.
- recalls and batches are linked through recall_batches; recall_recoveries reference the recall/batch association and may reference a warehouse, recipient and user.
- event_outbox stores event type, aggregate identity, JSON payload, delivery status and retry/processing metadata.

6. Relational Schema and Data Dictionary

The schema below is summarized from the repository's StockSync_Database_Setup.sql. PK = primary key; FK = foreign key; UK = unique key. See Appendix B for enlarged images of the schema supplied in the original draft. Use the actual SQL file for the complete column definitions, types, nullability and constraints.

Table	Key attributes (summary)	Purpose
app_users	user_id (PK); email (UK); password_hash; role; is_active	Accounts and role assignments
products	product_id (PK); sku (UK); product_name; category; unit_of_measure; reorder_level	Product master
suppliers	supplier_id (PK); supplier details and status	Supplier master
batches	batch_id (PK); batch_code (UK); product_id (FK); supplier_id (FK); manufacture_date; expiry_date; unit_cost; status	Supplier batch traceability
warehouses	warehouse_id (PK); warehouse_code (UK); warehouse_name; address; city; capacity_units; status	Warehouse locations
inventory	inventory_id (PK); batch_id (FK); warehouse_id (FK); quantity_on_hand; quantity_reserved; quantity_quarantined; available_quantity (generated); UK(batch_id,warehouse_id)	Stock balances by batch and warehouse
stock_reservations	reservation_id (PK); batch_id (FK); warehouse_id (FK); quantity; reserved_by (FK); status; expiry/notes timestamps as defined in SQL	Reservation lifecycle
stock_transfers	transfer_id (PK); batch_id (FK); source_warehouse_id (FK); destination_warehouse_id (FK); quantity; status; user reference	Warehouse movement
recipients	recipient_id (PK); recipient/contact/organization/address fields	Shipment destinations
shipments	shipment_id (PK); shipment_code (UK); recipient_id (FK); source_warehouse_id (FK); status; dispatch/delivery timestamps; created_by (FK)	Shipment lifecycle
shipment_items	shipment_item_id (PK); shipment_id (FK); batch_id (FK); quantity_dispatched; quantity_received; discrepancy_notes; UK(shipment_id,batch_id)	Batch quantities per shipment
defect_reports	defect_id (PK); defect_code (UK); batch_id (FK); defect_type; description; affected_quantity; severity/status; reporter/reviewer references	Quality reporting
recalls	recall_id (PK); recall_code (UK); title; reason; severity; status; initiated_by (FK); started_at; completed_at	Recall case
recall_batches	composite PK (recall_id,batch_id); both columns are FKs; affected_quantity; quarantine_quantity; recovered_quantity	Batches affected by a recall
recall_recoveries	recovery_id (PK); recall_id and batch_id (composite FK to recall_batches); optional warehouse/recipient/user references; quantity; action_type	Recovery/quarantine actions
event_outbox	event_id (PK); event_type; aggregate_type/id; JSON payload; delivery status, attempts and timestamps	Reliable event handoff to MongoDB

7. Integrity, Indexing and Advanced SQL

- Primary keys and foreign keys connect products, batches, suppliers, warehouses, users, inventory, shipments and recalls.
- Unique constraints include email, SKU, batch code, warehouse code, shipment code, defect code and the batch/warehouse inventory combination.
- CHECK constraints validate inventory quantity bounds, positive dispatched quantities, valid batch date order, positive reservation/transfer quantities and distinct transfer warehouses where defined in SQL.
- inventory.available_quantity is a stored generated value calculated as quantity_on_hand − quantity_reserved − quantity_quarantined.
- Reporting views in the SQL setup include vw_inventory_overview, vw_low_stock, vw_recall_recipient_trace and vw_shipment_discrepancies.
- Stored procedures in the SQL setup include sp_cancel_reservation and sp_record_quarantine. Confirm other procedure names against the imported database before presenting them.

- Triggers trg_inventory_insert_outbox and trg_inventory_update_outbox insert event records after inventory insert/update operations.
- Procedures use transactions, row locking (SELECT ... FOR UPDATE) and error handlers for critical multi-step stock and quarantine operations.

8. MongoDB Model and System Architecture

MongoDB database: inventory_events; collection: events. Each document represents an event and carries event-type-specific payload and aggregate metadata. Inventory triggers write to the MySQL event_outbox. A synchronization API processes pending rows, writes documents to MongoDB and updates the delivery status in MySQL. This outbox approach reduces the chance of losing an event when an inventory transaction commits.

Architecture: Browser UI (Next.js/React) → Next.js API handlers → MySQL operational tables/views/procedures. The event synchronization handler additionally writes event documents to MongoDB. Authentication uses password hashing and signed sessions as documented; Zod is used for input validation. Credentials are configured through environment variables and must not be committed to Git.

9. Application Modules

Route	Purpose
/	Dashboard and operational summary
/login	Authentication
/products	Product management
/inventory	Inventory overview
/warehouses	Warehouse information
/transfers	Stock transfers
/reservations	Stock reservations
/shipments	Shipment tracking
/defects	Defect reporting and review
/recalls	Recall management and recovery
/recipients	Recipient traceability

10. Testing and Results

The application was validated through code-quality checks, production build verification, database connectivity checks and event synchronization checks. Functional testing covers inventory management, reservations, shipment workflows, recall operations, database integrity and role-based access.

Test ID	Test	Validation
T01	Lint & production build	Execute npm run lint and npm run build to verify code quality and production compilation.
T02	Reservation lifecycle	Create and cancel a reservation; verify the corresponding inventory quantities.
T03	Shipment status validation	Test valid shipment status transitions and confirm invalid transitions are rejected.
T04	Recall/quarantine	Verify recall-batch associations, quarantine actions and recovery records.
T05	MySQL-to-MongoDB synchronization	Run the event worker and compare MySQL outbox delivery records with MongoDB event documents.
T06	Authorization	Verify permitted and denied operations for the configured user roles.
T07	Database integrity	Test applicable constraints, invalid quantities, foreign-key rules and transaction error handling.
T08	SQL query requirements	Demonstrate implemented joins, aggregates, grouping, subqueries and CTEs where applicable.

11. Security, Setup and Limitations

- Install Node.js/npm, MySQL and MongoDB; clone the public repository and install dependencies.
- Copy .env.example to .env.local and configure database connection variables locally. Never commit actual credentials.
- Review StockSync_Database_Setup.sql before executing it. It contains DROP TABLE statements and sample data; import only into a fresh disposable development database.
- The SQL dump contains DEFINER clauses such as root@localhost. Adjust privileges/definers if required by the local MySQL account.
- After import, verify that all expected tables, constraints, indexes, views, triggers and stored procedures exist.
- The current report does not claim vector search; MongoDB is used as a document/event store.

Note: Screenshots are available in screenshots folder under docs folder

12. Team Contributions

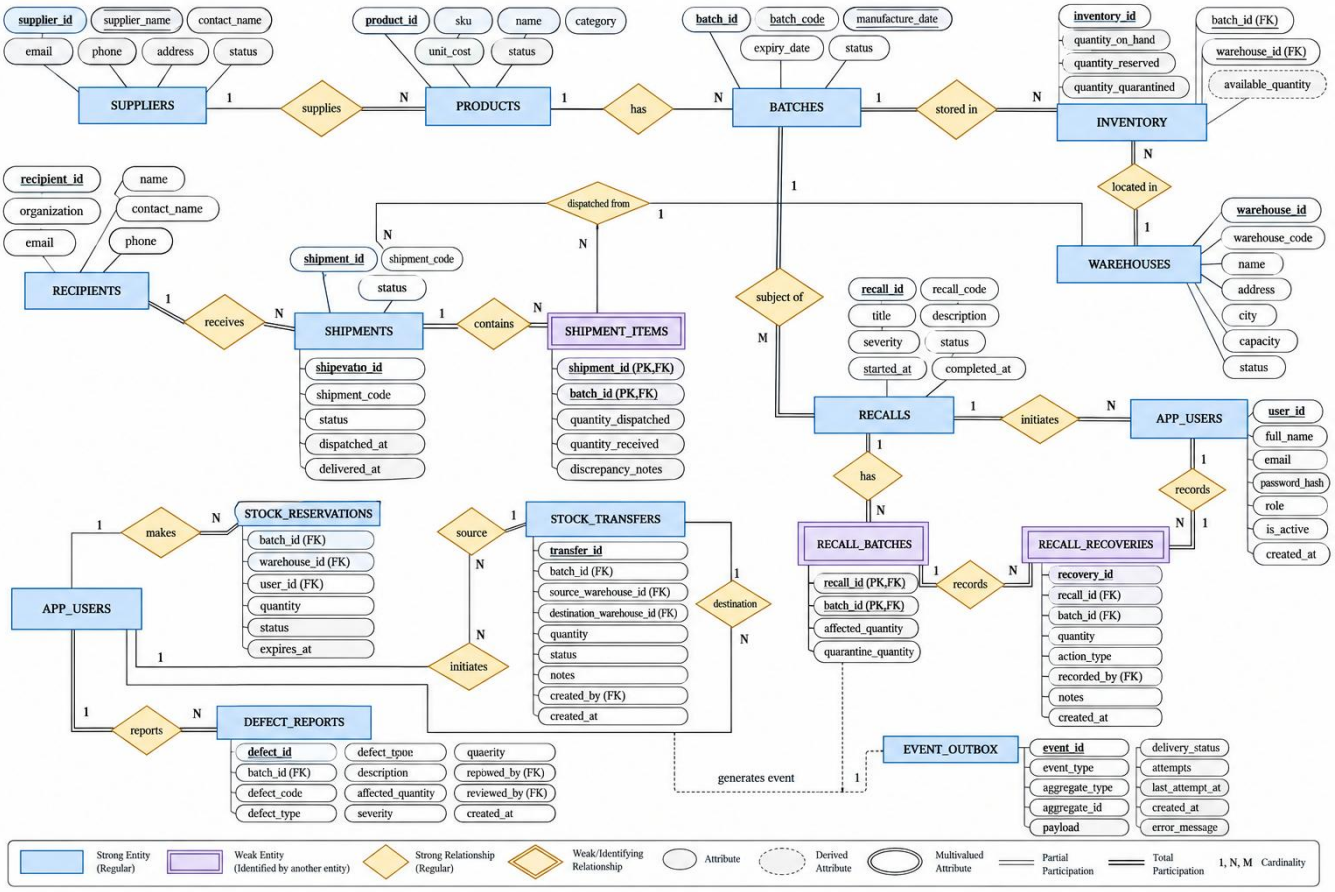
Both team members contributed to the planning, development, integration, testing and documentation of StockSync. Responsibilities were distributed across database development, application implementation and system validation.

Work Area	**Rekha Dhorigol**	**Reet Porwal**
Requirements and system planning	Co-defined project objectives, functional requirements and inventory workflows.	Co-defined project objectives, functional requirements and inventory workflows.
Database design and ER diagram	Led relational entity design, relationship definition and ER diagram preparation.	Reviewed entity relationships and their consistency with application requirements.
Relational schema and data dictionary	Prepared and reviewed table structures, primary keys, foreign keys and relational constraints.	Contributed to schema validation and documentation of table purposes and relationships.
SQL implementation and advanced features	Focused on relational constraints, reporting views and SQL query development.	Focused on stored procedures, triggers, transactional workflows and event-outbox integration.
Frontend and application modules	Contributed to inventory, warehouse and stock workflow requirements and verification.	Contributed to application pages, API integration and operational module implementation.
Authentication and validation	Contributed to role requirements and verification of access-controlled workflows.	Contributed to authentication/session integration and request validation.
MongoDB and event synchronization	Contributed to event data requirements and verification of outbox records.	Contributed to worker configuration, database connectivity and MySQL–MongoDB synchronization.
Testing and debugging	Tested database integrity and inventory-related workflows and helped resolve integration issues.	Tested application build, API behavior and event-worker execution and helped resolve integration issues.
Report and demonstration	Co-prepared the project report, database explanations and demonstration workflow.	Co-prepared the project report, repository documentation and demonstration workflow.

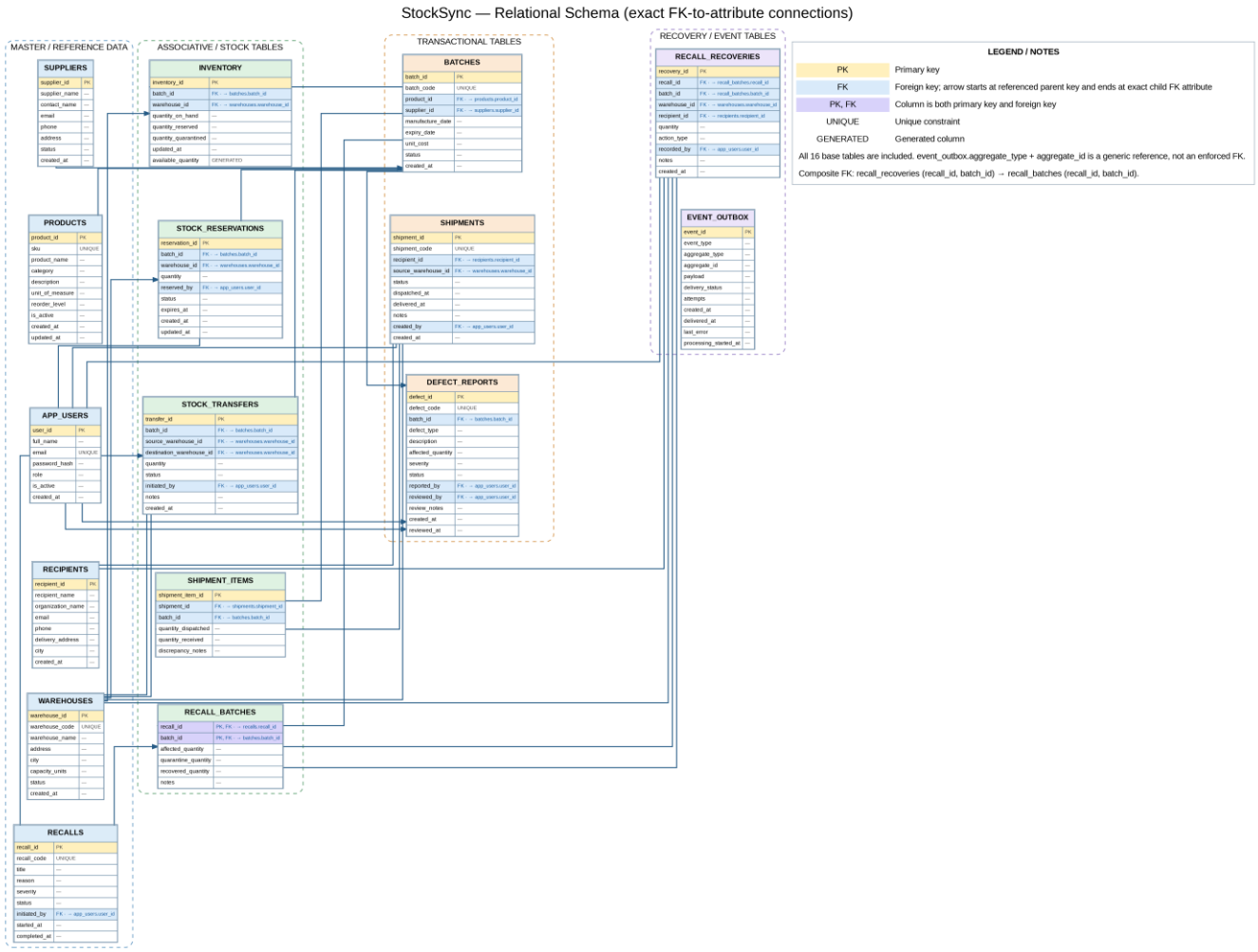
13. Conclusion

StockSync connects inventory management with batch traceability and recall recovery. Its relational model covers products, suppliers, batches, warehouses, stock, shipments, recipients, defects and recalls. MySQL supports structured relationships and integrity, while MongoDB stores event documents synchronized through a MySQL outbox. The project goes beyond basic CRUD by including stock reservation/transfer workflows, shipment status validation, recall quarantine/recovery, reporting views, stored procedures, triggers and event synchronization. Final readiness depends on confirming all features against the live repository, completing the test evidence, adding UI/database screenshots, and documenting each team member's actual contribution.

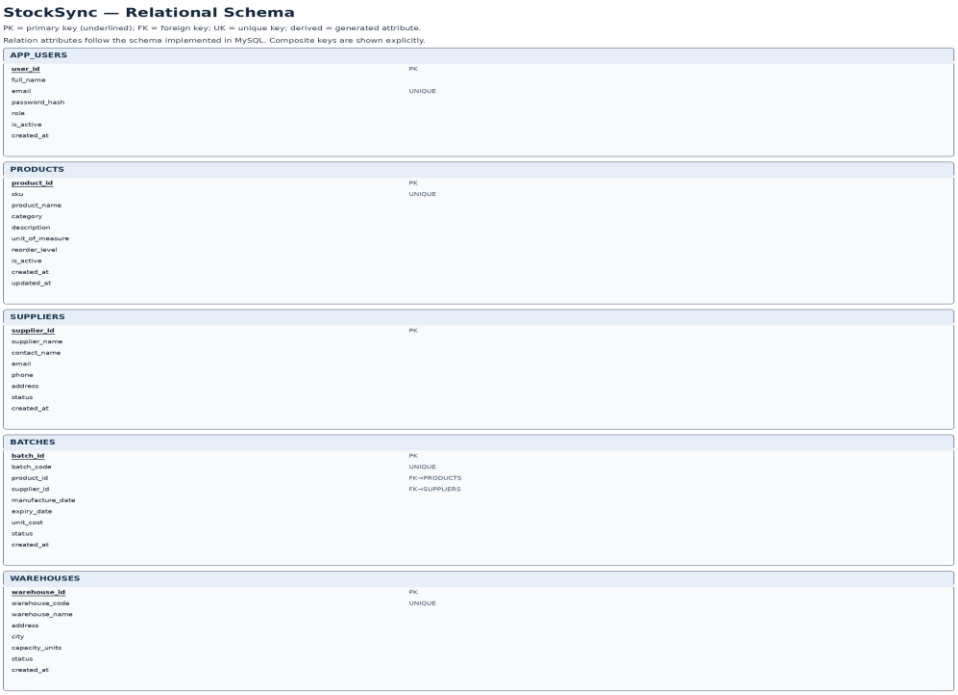
Appendix A. ER Diagram



Appendix B. Relational Schema



For better clarity:



StockSync — Relational Schema

PK = primary key (underlined); FK = foreign key; UK = unique key; derived = generated attribute.
Relation attributes follow the schema implemented in MySQL. Composite keys are shown explicitly.

APP_USERS

user_id	PK
full_name	
email	UNIQUE
password_hash	
role	
is_active	
created_at	

PRODUCTS

product_id	PK
sku	UNIQUE
product_name	
category	
description	
unit_of_measure	
reorder_level	
is_active	
created_at	
updated_at	

SUPPLIERS

supplier_id	PK
supplier_name	
contact_name	
email	
phone	
address	
status	
created_at	

BATCHES

batch_id	PK
batch_code	UNIQUE
product_id	FK→PRODUCTS
supplier_id	FK→SUPPLIERS
manufacture_date	
expiry_date	
unit_cost	
status	
created_at	

WAREHOUSES

warehouse_id	PK
warehouse_code	UNIQUE
warehouse_name	
address	
city	
capacity_units	
status	
created_at	

INVENTORY

inventory_id	PK
batch_id	FK→BATCHES
warehouse_id	FK→WAREHOUSES
quantity_on_hand	
quantity_reserved	
quantity_quarantined	
available_quantity	DERIVED
updated_at	
(batch_id, warehouse_id)	UNIQUE

STOCK_RESERVATIONS

reservation_id	PK
batch_id	FK→BATCHES
warehouse_id	FK→WAREHOUSES
quantity	
reserved_by	FK→APP_USERS
status	
expires_at	
created_at	
updated_at	

STOCK_TRANSFERS

transfer_id	PK
batch_id	FK→BATCHES
source_warehouse_id	FK→WAREHOUSES
destination_warehouse_id	FK→WAREHOUSES
quantity	
status	
initiated_by	FK→APP_USERS
notes	
created_at	

RECIPIENTS

recipient_id	PK
recipient_name	
organization_name	
email	
phone	
delivery_address	
city	
created_at	

SHIPMENTS

shipment_id	PK
shipment_code	UNIQUE
recipient_id	FK→RECIPIENTS
source_warehouse_id	FK→WAREHOUSES
status	
dispatched_at	
delivered_at	
notes	
created_by	FK→APP_USERS
created_at	

SHIPMENT_ITEMS

shipment_item_id	PK
shipment_id	FK→SHIPMENTS
batch_id	FK→BATCHES
quantity_dispatched	
quantity_received	
discrepancy_notes	

DEFECT_REPORTS

defect_id	PK
defect_code	UNIQUE
batch_id	FK→BATCHES
defect_type	
description	
affected_quantity	
severity	
status	
reported_by	FK→APP_USERS
reviewed_by	FK→APP_USERS
review_notes	
created_at	
reviewed_at	

RECALLS

recall_id	PK
recall_code	UNIQUE
title	
reason	
severity	
status	
initiated_by	FK→APP_USERS
started_at	
completed_at	

RECALL_BATCHES

recall_id	PK, FK→RECALLS
batch_id	PK, FK→BATCHES
affected_quantity	
quarantine_quantity	
recovered_quantity	
notes	

RECALL_RECOVERIES

recovery_id	PK
(recall_id, batch_id)	composite FK→RECALL_BATCHES
warehouse_id	FK→WAREHOUSES
recipient_id	FK→RECIPIENTS
quantity	
action_type	
recorded_by	FK→APP_USERS
notes	
created_at	

EVENT_OUTBOX

event_id	PK
event_type	
aggregate_type	
aggregate_id	logical reference
payload	
delivery_status	
attempts	
created_at	
delivered_at	
last_error	